Werkzeuge · Warm-up-Karten

Warm-up-Karten Welt 5 – Der Wald

Kurze Papier-Aufgaben für den Einstieg jeder Doppelstunde („10' Papier-Warm-up, ohne Gerät"). Die Karten W5-01 bis W5-14 begleiten den Wald (DS 31–35): BinTree-Code ↔ Bild, Baumbegriffe, Traversierungen und Rekonstruktion, Termbäume, rekursive Baumalgorithmen mit Rekursionstabelle, Level-Order mit Schlange, Suchbäume (Einfügen, Suchen, Löschen), entartet vs. ausgeglichen und die isBST-Falle – alle mit den Testbäumen der Baumschule.

So funktioniert's

- Karte ziehen (oder zugeteilt bekommen), **ohne Gerät** auf einem eigenen Blatt lösen.
- Selbst- oder Partnerkontrolle mit dem Lösungsteil (gleiche Nummer). Ziel: Tracetabellen und Struktogramme ohne Notationsfehler.
- Die gleichen Karten gibt es digital im **Warm-up-Trainer von Algo Quest Lab** (Welt-Filter „Welt 5"; Leitner-Prinzip mit ehrlicher Selbsteinschätzung, ohne Punkte). Boss-Training vor Boss 5: Karten W5-05, W5-07, W5-08, W5-13 und W5-14 – im Lab zusätzlich der Online-Boss „Die drei Baumhüter – Online-Duell".

Achtung Abi:
Binärbaum exakt nach EH-Schnittstelle (BinTree). Ein nicht leerer Baum hat **immer zwei Teilbäume** (ggf. leer, nie null). Baum-Methoden schreibst du **rekursiv** mit dem Basisfall **leerer Baum** (`if (b.isEmpty())`) – getItem/getLeft/getRight auf einem leeren Baum sind Laufzeitfehler. **Casten:** `int w = (Integer) b.getItem();`. Höhe: leer 0, Blatt 1; Wurzel = Ebene 0; Rekursionstiefe des Erstaufrufs = 0.

Kartenübersicht

Nr.	Kategorie	Titel	Min.	Einsatz
W5-01	Objektkarten und Referenzen	Der Setzling	7	DS 31
W5-02	Methodenkopf und Signatur	Wegweiser im Wald	6	DS 31
W5-03	Funktionsweise erläutern	Drei Wege durch den Wald	8	DS 31
W5-04	Lückencode	Der Rechenbaum	8	DS 31
W5-05	Tracetabelle	Wie hoch ist der Baum?	8	DS 32 + Boss
W5-06	Fehler im Java-Code	Der Ast bricht	7	DS 32
W5-07	Code ↔ Struktogramm	Zählen auf Ebene k	8	DS 32 + Boss
W5-08	Methode implementieren	Blätter sammeln	8	DS 32 + Boss
W5-09	Tracetabelle	Die Wanderschlange	8	DS 33 (Kür)
W5-10	Funktionsweise erläutern	Die Item-Datenbank wächst	7	DS 34
W5-11	Struktogramm lesen	Suchen mit Zählwerk	7	DS 34
W5-12	Verändern und Erweitern	Ein Item verschwindet	8	DS 34
W5-13	Funktionsweise erläutern	Krumm oder buschig?	8	DS 35 + Boss
W5-14	Fehler im Java-Code	Die isBST-Falle	8	DS 35 + Boss

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht.

WARM-UP W5-01 Objektkarten und Referenzen

Welt 5 – Wald · ⏱ 7 min

Der Setzling

Bibliothek „Niedersachsen“: BinTree ist nicht generisch – getItem() liefert Object und wird gecastet, z. B. (Integer) b.getItem(). Ein nicht leerer Baum hat immer zwei Teilbäume (ggf. leere Bäume, nie null). getItem, getLeft und getRight auf einem leeren Baum lösen einen Laufzeitfehler aus.

a) Führe die Zeilen 1 bis 10 aus und zeichne den Baum b danach (Knoten als Kreise, leere Teilbäume nicht zeichnen).

b) Gib für deinen Baum die Wurzel, alle Blätter, alle inneren Knoten, die Höhe und die Ebene des Knotens 7 an (Wurzel = Ebene 0; Höhe: leerer Baum 0, Blatt 1).

c) Erläutere, warum Zeile 6 keinen Fehler auslöst, obwohl dort noch kein Knoten steht, und warum die Zeilen 8 und 9 den Baum b verändern, obwohl sie nur t verwenden.

d) Gib an, was die Anweisung `int x = (Integer) b.getLeft().getLeft().getLeft().getItem();` nach Zeile 10 bewirkt.

```
1  BinTree b = new BinTree(8);
2  b.setLeft(new BinTree(3));
3  b.setRight(new BinTree(10));
4  b.getLeft().setLeft(new BinTree(1));
5  b.getLeft().setRight(new BinTree(6));
6  b.getRight().getRight().setItem(14);
7  BinTree t = b.getLeft().getRight();
8  t.setLeft(new BinTree(4));
9  t.setRight(new BinTree(7));
10 b.getRight().getRight().setLeft(new BinTree(13));
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-01

WARM-UP W5-02 Methodenkopf und Signatur

Welt 5 – Wald · ⏱ 6 min

Wegweiser im Wald

Die Methoden der Klasse Rekursiv sind in der Notation des EH angegeben (alle static).

a) Gib zu jeder Signatur den Java-Methodenkopf an:

(1) maximum(b: BinTree vom Inhaltstyp Ganzzahl): Ganzzahl (2) pfadZu(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Zeichenkette (3) blaetter(b: BinTree vom Inhaltstyp Ganzzahl): dynamische Reihung vom Inhaltstyp Ganzzahl (4) istGleich(a: BinTree, b: BinTree): Wahrheitswert

b) Gib umgekehrt die Signaturen zu `public static void spiegeln(BinTree b)` und `public static BinTree baueAus(int[] r)` in EH-Notation an.

c) b ist ein nicht leerer BinTree vom Inhaltstyp Ganzzahl, t einer vom Inhaltstyp Zeichenkette. Entscheide für jede der Zeilen 1 bis 6, ob der Compiler sie akzeptiert, und korrigiere fehlerhafte Zeilen. Welche akzeptierte Zeile kann trotzdem zur Laufzeit scheitern?

```
1  int w = b.getItem();
2  int v = (Integer) b.getItem() + 1;
3  int u = (Integer) b.getLeft().getItem();
4  String s = (String) t.getItem();
5  boolean plus = ((String) t.getItem()).equals("+");
6  int z = (Integer) b.getLeft();
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-02

WARM-UP W5-03 Funktionsweise erläutern

Welt 5 – Wald · ⏱ 8 min

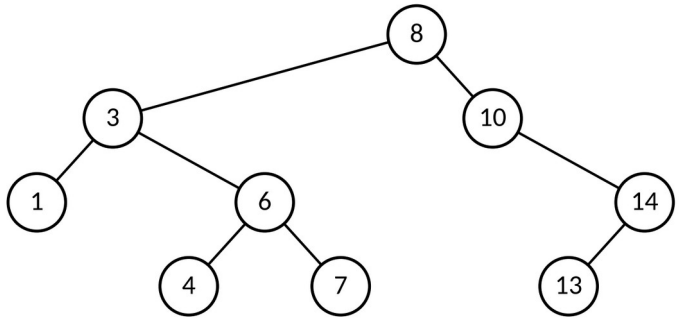
Drei Wege durch den Wald

Traversierungen: Preorder = Wurzel, links, rechts · Inorder = links, Wurzel, rechts · Postorder = links, rechts, Wurzel (jeweils rekursiv für jeden Teilbaum).

a) Gib Preorder, Inorder und Postorder des Baums A an.

b) Von einem Baum B kennt man nur die Preorder 5 9 2 7 4 9 1 und die Inorder 2 9 7 5 4 1 9. Rekonstruiere B schrittweise, zeichne ihn und gib zur Kontrolle seine Postorder an. Tipp: Das erste Element der Preorder ist die Wurzel; in der Inorder stehen links davon die Werte des linken, rechts davon die des rechten Teilbaums. (Die 9 kommt zweimal vor – das stört hier nicht, weil sie in verschiedenen Teilbäumen liegt.)

c) Zeige mit zwei Bäumen aus je zwei Knoten, dass Preorder und Postorder allein einen Baum nicht eindeutig festlegen.



Baum A

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-03

WARM-UP W5-04 Lückencode

Welt 5 – Wald · ⏱ 8 min

Der Rechenbaum

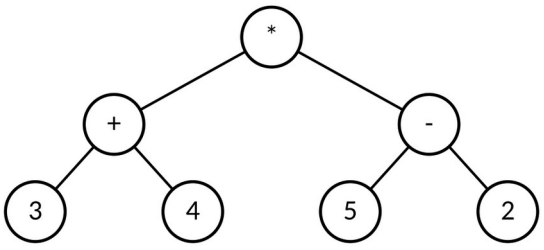
Ein Termbaum enthält Zeichenketten: Blätter sind Zahlen ("3", "12"), innere Knoten die Operatoren "+", "-", "*", "/" (ganzzahlig). Der Baum T stellt den Term (3 + 4) * (5 − 2) dar.

a) Gib Preorder, Inorder und Postorder von T an. Welche Traversierung liefert die Eingabe für den Postfix-Rechner aus Welt 4? Warum braucht die Inorder Klammern?

b) Zeichne den Termbaum zum Term 8 − 2 * 3 (Punkt vor Strich) und gib seinen Wert an.

c) Ergänze die Lücken (1) bis (5) der Methode berechne(t: BinTree vom Inhaltstyp Zeichenkette): Ganzzahl. t ist ein nicht leerer, korrekter Termbaum; zahl(s) wandelt eine Ziffernfolge in eine Ganzzahl um (vorgegeben).

```
public static int berechne(BinTree t) {
    String s = __(1)__ t.getItem();
    if (__(2)__) {
        return zahl(s);
    }
    int l = berechne(__(3)__);
    int r = berechne(t.getRight());
    if (s.equals("+")) { return l + r; }
    if (__(4)__) { return l - r; }
    if (s.equals("*")) { return l * r; }
    return __(5)__;
}
```



Baum T

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-04

WARM-UP W5-05 Tracetabelle

Welt 5 – Wald · ⏱ 8 min

Wie hoch ist der Baum?

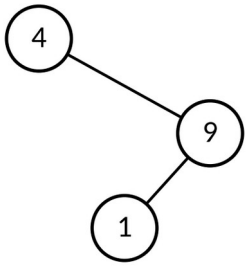
Die Methode hoehe ist rekursiv; Basisfall ist der leere Baum (Höhe 0). Der Baum C ist der rechte Teilbaum des zahlenbaums.

a) Dokumentiere den Aufruf hoehe(C) in einer Rekursionstabelle mit den Spalten Rekursionstiefe, Aufruf und Rückgabewert (Erstaufruf = Tiefe 0). Trage die Aufrufe in der Reihenfolge ein, in der sie beginnen; leere Teilbäume schreibst du als hoehe(leer).

b) Gib an, wie oft hoehe für einen Baum mit n Knoten insgesamt aufgerufen wird, und begründe. Wie oft ist es beim ganzen zahlenbaum (7 Knoten)?

c) Ändere den Rumpf so ab, dass die Methode die Anzahl der Blätter liefert.

```
public static int hoehe(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    int l = hoehe(b.getLeft());
    int r = hoehe(b.getRight());
    if (l > r) {
        return l + 1;
    }
    return r + 1;
}
```



Baum C

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-05

WARM-UP W5-06 Fehler im Java-Code

Welt 5 – Wald · ⏱ 7 min

Der Ast bricht

Die Methode summe(b: BinTree vom Inhaltstyp Ganzzahl): Ganzzahl soll die Summe aller Werte liefern (leerer Baum: 0). Bibliothek „Niedersachsen“: BinTree ist nicht generisch – getItem() liefert Object und wird gecastet, z. B. (Integer) b.getItem().

a) Nenne die Zeilen, die der Compiler nicht akzeptiert, und korrigiere sie.

b) Nach der Korrektur aus a) wird summe mit dem zahlenbaum aufgerufen. Gib die Aufrufe in der Reihenfolge an, in der sie beginnen, bis es zum Laufzeitfehler kommt. Erläutere die Ursache.

c) Korrigiere die Methode so, dass sie für jeden Baum (auch den leeren) funktioniert, und gib summe(zahlenbaum) an.

```
1  public static int summe(BinTree b) {
2      if (b.isLeaf()) {
3          return b.getItem();
4      }
5      return b.getItem() + summe(b.getLeft()) + summe(b.getRight());
6  }
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-06

WARM-UP W5-07 Code ↔ Struktogramm

Welt 5 – Wald · ⏱ 8 min

Zählen auf Ebene k

Die Methode anzahlAufEbene liefert die Anzahl der Knoten auf der angegebenen Ebene (Wurzel = Ebene 0).

a) Gib die Rückgabewerte für den zahlenbaum und die Ebenen 0, 1, 2, 3 und 4 an.

b) Entwickle zur Methode ein Struktogramm nach den Abi-Konventionen (Kopf mit Signatur, ←, „gib … zurück“; den rekursiven Aufruf notierst du als Anweisung, z. B. l ← anzahlAufEbene(…)).

c) Jemand vertauscht die beiden if-Anweisungen (erst ebene == 0, dann isEmpty). Gib an, welche Werte sich für den zahlenbaum dann bei den Ebenen 2 und 4 ergeben, und erläutere den Fehler.

```
public static int anzahlAufEbene(BinTree b, int ebene) {
    if (b.isEmpty()) {
        return 0;
    }
    if (ebene == 0) {
        return 1;
    }
    int l = anzahlAufEbene(b.getLeft(), ebene - 1);
    int r = anzahlAufEbene(b.getRight(), ebene - 1);
    return l + r;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-07

WARM-UP W5-08 Methode implementieren

Welt 5 – Wald · ⏱ 8 min

Blätter sammeln

Für die Item-Datenbank sollen die Blattwerte eines Baums gesammelt werden.

a) Implementiere blaetter(b: BinTree vom Inhaltstyp Ganzzahl): dynamische Reihung vom Inhaltstyp Ganzzahl. Die Methode liefert eine neue dynamische Reihung mit allen Blattwerten von links nach rechts (leerer Baum: leere Reihung). Nutze dazu – wie im L5-Gerüst – die überladene rekursive Hilfsmethode blaetter(b: BinTree, d: dynamische Reihung): nichts (gleicher Name, zusätzlicher Parameter).

b) Gib das Ergebnis für den zahlenbaum und für den beispielBST an.

c) Erläutere, warum die Hilfsmethode eine dynamische Reihung als Parameter bekommt, statt in jedem Aufruf eine neue zu erzeugen.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-08

WARM-UP W5-09 Tracetabelle

Welt 5 – Wald · ⏱ 8 min

Die Wanderschlange

Die Methode levelorder durchläuft einen Baum ebenenweise mithilfe einer Schlange vom Inhaltstyp BinTree. In der Schlange liegen also Teilbäume, nicht Zahlen.

a) Führe levelorder(beispielBST) aus. Notiere an jeder mit „// → Tabellenzeile“ markierten Stelle den entnommenen Knoten, die Schlange (vorn zuerst, je Teilbaum sein Wurzelwert) und s.

b) Erläutere, warum in Zeile 8 gecastet werden muss.

c) Die Schlange wird durch einen Stapel ersetzt (push statt enqueue, pop statt dequeue). Gib die Ausgabe für den beispielBST an. Welche Reihenfolge entsteht, wenn man zusätzlich die beiden if-Anweisungen vertauscht?

```
1  public static String levelorder(BinTree b) {
2      String s = "";
3      Queue q = new Queue();
4      if (!b.isEmpty()) {
5          q.enqueue(b);
6      }
7      while (!q.isEmpty()) {
8          BinTree t = (BinTree) q.dequeue();
9          s = s + t.getItem() + " ";
10         if (!t.getLeft().isEmpty()) {
11             q.enqueue(t.getLeft());
12         }
13         if (!t.getRight().isEmpty()) {
14             q.enqueue(t.getRight());
15         }   // → Tabellenzeile
16     }
17     return s;
18 }
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-09

WARM-UP W5-10 Funktionsweise erläutern

Welt 5 – Wald · ⏱ 7 min

Die Item-Datenbank wächst

In der Item-Datenbank liegen die Item-Nummern in einem binären Suchbaum (BST): Für jeden Knoten sind alle Werte im linken Teilbaum kleiner, alle im rechten größer. Duplikate werden ignoriert. Eingefügt wird immer als neues Blatt am Ende des Suchwegs.

a) Füge in einen leeren BST nacheinander 8, 3, 10, 1, 6, 14, 4, 7, 13 ein und zeichne ihn.

b) Füge anschließend 5 und 12 ein. Gib jeweils den Suchweg (verglichene Knoten) an und markiere die neuen Knoten.

c) Gib die Inorder des Baums aus b) an und begründe, warum die Inorder eines BST immer aufsteigend sortiert ist. Was passiert beim Einfügen von 6?

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-10

WARM-UP W5-11 **Struktogramm lesen**

Welt 5 – Wald · ⏱ 7 min

Suchen mit Zählwerk

Die Operation vergleiche zählt, wie viele (nicht leere) Knoten bei der Suche nach x im BST b untersucht werden. Die Inhalte sind Ganzzahlen.

a) Gib für den beispielBST die untersuchten Knoten und die Rückgabe für x = 7, x = 5, x = 13 und x = 8 an.

b) Gib den größtmöglichen Rückgabewert für diesen Baum an und begründe, wovon er abhängt.

c) Erläutere, warum je Schritt nur ein Teilbaum untersucht werden muss. Wie viele Knoten müsste man im schlimmsten Fall untersuchen, wenn der Baum kein Suchbaum wäre (z. B. der zahlenbaum)?

```
vergleiche(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Ganzzahl
  anzahl ← 0
  t ← b
  solange nicht t.isEmpty()
    anzahl ← anzahl + 1
    w ← t.getItem()
    falls x = w
      dann: gib anzahl zurück
      sonst: falls x < w
               dann: t ← t.getLeft()
               sonst: t ← t.getRight()
  gib anzahl zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-11

WARM-UP W5-12 **Verändern und Erweitern**

Welt 5 – Wald · ⏱ 8 min

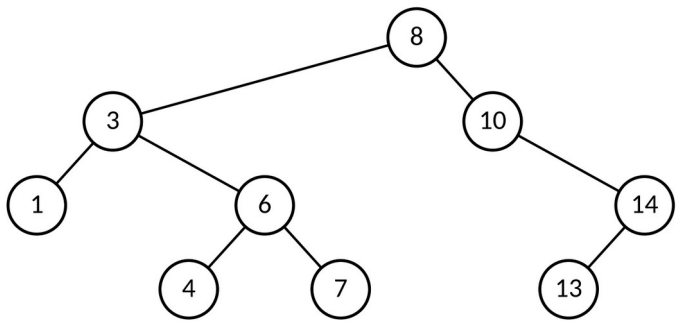
Ein Item verschwindet

Löschen im BST – drei Fälle: (1) Blatt: einfach entfernen. (2) Knoten mit einem Kind: Das Kind (mit seinem ganzen Teilbaum) rückt an die Stelle des Knotens. (3) Knoten mit zwei Kindern: Der Wert wird durch den Nachfolger ersetzt (kleinster Wert im rechten Teilbaum: einmal rechts, dann ganz nach links); der Nachfolger wird dort nach Fall 1 oder 2 gelöscht.

Lösche jeweils aus dem ursprünglichen beispielBST und zeichne den Baum danach:

a) 7 b) 10 c) 3 d) 8 (die Wurzel)

e) Statt des Nachfolgers darf man auch den Vorgänger nehmen (größter Wert im linken Teilbaum). Gib an, welcher Wert in c) und d) dann aufrückt. Begründe, warum der Nachfolger höchstens ein Kind hat.



Baum beispielBST

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-12

WARM-UP W5-13 Funktionsweise erläutern

Welt 5 – Wald · ⏱ 8 min

Krumm oder buschig?

Die Methode ausgeglichenAus baut aus einer aufsteigend sortierten Reihung einen BST: Die Mitte wird Wurzel, aus den Hälften links und rechts davon entstehen rekursiv die Teilbäume. Aufruf: ausgeglichenAus(r, 0, r.length − 1) – im L5-Gerüst erledigt das ausgeglichenAus(sortiert).

a) Zeichne den BST, der durch Einfügen von 1, 2, 3, 4, 5 (in dieser Reihenfolge) entsteht, und den Baum ausgeglichenAus für r = {1, 2, 3, 4, 5}. Gib jeweils die Höhe an.

b) Die maximale Anzahl der Vergleiche bei einer Suche ist die Höhe. Gib sie für n = 7, 15, 31 und 1023 Werte an – einmal für den entarteten Baum, einmal für den Baum aus ausgeglichenAus. Was bewirkt eine Verdopplung von n jeweils?

c) Erläutere, warum die Item-Datenbank entartet, wenn neue Items mit aufsteigender Nummer eingefügt werden, und wie man Abhilfe schaffen kann.

```
public static BinTree ausgeglichenAus(int[] r, int lo, int hi) {
    BinTree b = new BinTree();
    if (lo <= hi) {
        int mitte = (lo + hi) / 2;
        b.setItem(r[mitte]);
        b.setLeft(ausgeglichenAus(r, lo, mitte - 1));
        b.setRight(ausgeglichenAus(r, mitte + 1, hi));
    }
    return b;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-13

WARM-UP W5-14 Fehler im Java-Code

Welt 5 – Wald · ⏱ 8 min

Die isBST-Falle

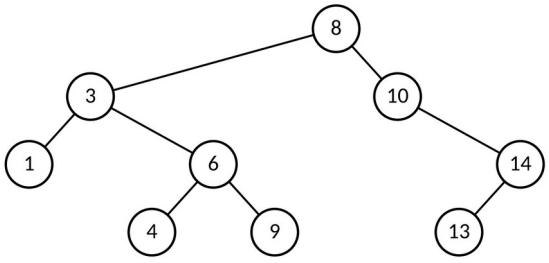
istBSTfalsch soll prüfen, ob b ein binärer Suchbaum ist (alle Werte im linken Teilbaum kleiner, alle im rechten größer als der Wert des Knotens – für jeden Knoten).

a) Gib die Rückgabe von istBSTfalsch für den Baum D an und begründe, dass D kein BST ist.

b) Erläutere den Denkfehler der Methode.

c) Korrigiere: Implementiere istBST(b: BinTree vom Inhaltstyp Ganzzahl): Wahrheitswert mit der Hilfsmethode istBST(b, min, max), die prüft, ob alle Werte echt zwischen min und max liegen. Alle Werte sind ≥ 0, deshalb steht −1 für „keine Grenze“. Gib die Grenzen min und max an, mit denen die Knoten auf dem Weg 8 → 3 → 6 → 9 geprüft werden.

```
public static boolean istBSTfalsch(BinTree b) {
    if (b.isEmpty()) {
        return true;
    }
    int w = (Integer) b.getItem();
    if (!b.getLeft().isEmpty()
            && (Integer) b.getLeft().getItem() >= w) {
        return false;
    }
    if (!b.getRight().isEmpty()
            && (Integer) b.getRight().getItem() <= w) {
        return false;
    }
    return istBSTfalsch(b.getLeft())
            && istBSTfalsch(b.getRight());
}
```



Baum D

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W5-14